

ATLAS

Unified Infrastructure Monitoring and Security Platform

PROJECT REPORT

Submitted in partial fulfillment of the academic requirements
for the Master of Computer Applications (MCA)

M V JESHWANTH VARMA

Roll No: 24914120063

KL CDOE, VJA Campus, India
2024-2026

ABSTRACT

Modern Linux systems expose operational information through multiple commands, logs, services, and interfaces. ATLAS is a web-based infrastructure monitoring, security, and administration platform developed to centralize essential host information in a single application. The implemented project combines a Node.js and Express.js backend, PostgreSQL 16, Firebase Authentication with Firebase Admin SDK, an HTML/CSS/JavaScript frontend, Chart.js-based presentation support, and a monitoring engine operating on Ubuntu 24.04 under WSL2. The backend provides REST endpoints for system information, CPU, memory, disk, and network monitoring and retrieves actual host data using Node.js operating-system functions, the systeminformation package, and Linux commands where required.

Authentication supports email/password and Google sign-in through Firebase. Backend middleware verifies Firebase identity tokens and an authentication service synchronizes verified users with PostgreSQL. The relational schema contains roles, users, sessions, servers, services, system_metrics, network_metrics, alerts, logs, audit_logs, and settings, with Admin, Operator, and Viewer roles seeded for application-level authorization design. Development testing verified PostgreSQL connectivity, Express server startup, static frontend delivery, Firebase login, backend token verification, user synchronization, and real monitoring API responses. ATLAS therefore demonstrates practical integration of operating-system monitoring, full-stack web development, database engineering, authentication, security middleware, and modular software architecture.

Keywords—System Monitoring; Cyber Security; Node.js; Express.js; PostgreSQL; Firebase Authentication; REST API; RBAC; Ubuntu; WSL2; Chart.js.

I. INTRODUCTION

Servers are central to modern applications, databases, communication systems, and cloud workloads. Their reliability depends on continuous visibility into processor utilization, memory consumption, storage availability, network activity, service state, and security events. Linux provides powerful native tools for observing these resources, but administrators often need to move among several commands and interfaces. A centralized

monitoring platform reduces this fragmentation and provides a consistent view of system health.

ATLAS was designed as a practical MCA project that combines infrastructure monitoring with secure web access and persistent application data. Rather than displaying hard-coded values, the monitoring backend collects information from the Linux host and exposes it through REST APIs. The frontend can consume these APIs to present operational information through dashboard cards and charts. Authentication is separated from monitoring logic, and database access is separated through service and repository layers, creating a modular structure that can be extended with additional monitoring and security features.

A. Problem Statement

System administrators require fast access to resource and security information, yet native operating-system data is distributed across commands, logs, and configuration utilities. A lightweight centralized platform is therefore required to collect host metrics, protect access to administrative information, store application records, and provide an extensible interface for future alerting, reporting, and security analysis.

B. Objectives

The objectives of ATLAS are to build a secure web-based monitoring platform; collect real CPU, memory, disk, network, uptime, and host information; expose monitoring data through structured REST APIs; implement user authentication and server-side identity verification; maintain application users and roles in PostgreSQL; organize the backend using modular controllers, services, repositories, middleware, and routes; and establish a foundation for security monitoring, alerts, reports, and real-time telemetry.

C. Scope

The current implementation focuses on a Linux host running Ubuntu 24.04 under WSL2. Core monitoring APIs, authentication integration, database connectivity, user synchronization, frontend delivery, and modular backend organization are implemented. Advanced remote-agent monitoring, automated incident response, complete historical analytics, production deployment through Nginx/Docker, and broader automation are treated as future extensions rather than completed features.

II. TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

Layer / Purpose	Technology	Use in ATLAS
Operating system	Ubuntu 24.04 LTS on WSL2	Linux execution and monitoring environment
Backend	Node.js v24.18.0, Express.js	REST API and application logic
Database	PostgreSQL 16	Users, roles, metrics, logs and settings
Authentication	Firebase Authentication + Firebase Admin SDK	Client identity and backend token verification
Frontend	HTML5, CSS3, JavaScript	Login, registration and dashboard interface
Visualization	Chart.js, Font Awesome, Google Fonts	Dashboard presentation
Monitoring	Node.js os, systeminformation, Linux commands	Real host metric collection
Security middleware	helmet, cors, express-validator, dotenv	HTTP and configuration protection
Utilities	morgan, cookie-parser	Request logging and request support
Version control	Git + GitHub	Source management and project history

The backend dependencies also include pg for PostgreSQL connectivity and Socket.IO as a dependency available for future or extended real-time communication. bcrypt and jsonwebtoken are present as backend security dependencies, while the implemented authentication flow described in this report uses Firebase identity tokens.

III. SYSTEM ANALYSIS AND DESIGN

A. Layered Architecture

ATLAS separates presentation, backend processing, identity verification, host monitoring, and persistence responsibilities. The browser handles user interaction. Express provides HTTP routing and application logic. Firebase provides user authentication, while Firebase

Admin verifies identity on the server. The monitoring module communicates with Ubuntu and supporting Node.js libraries to obtain system data. PostgreSQL stores application records and role information.

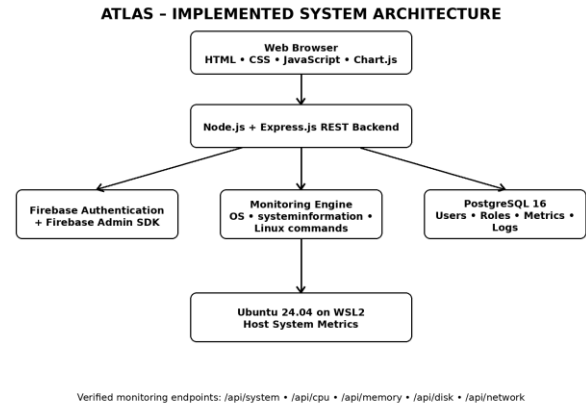


Fig. 1. Implemented ATLAS system architecture.

B. Project Organization

The project is organized into backend, frontend, database, design/documentation, scripts/tests, and project-level files. Inside backend/src, configuration, controllers, middleware, models, modules, repositories, routes, services, app.js, and server.js divide responsibilities. Monitoring and security are represented as feature modules, allowing future functionality to be introduced without placing all logic in a single application file.

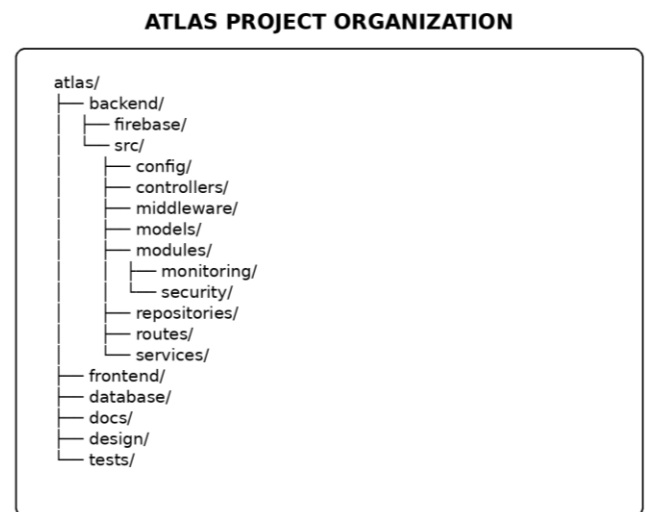


Fig. 2. ATLAS project organization.

C. Data Flow

A monitoring request reaches an Express route, which invokes the appropriate controller and monitoring service. The service obtains data from the operating system or supporting libraries and returns structured data to the API layer. The browser then consumes the JSON response. This design keeps operating-system collection logic separate from presentation logic.

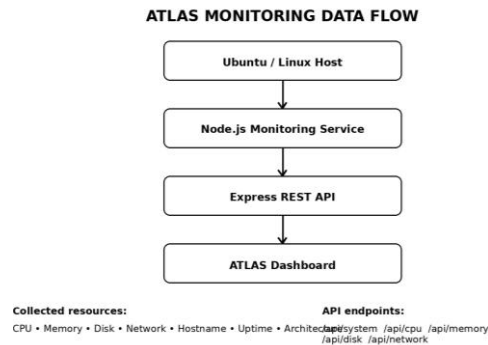
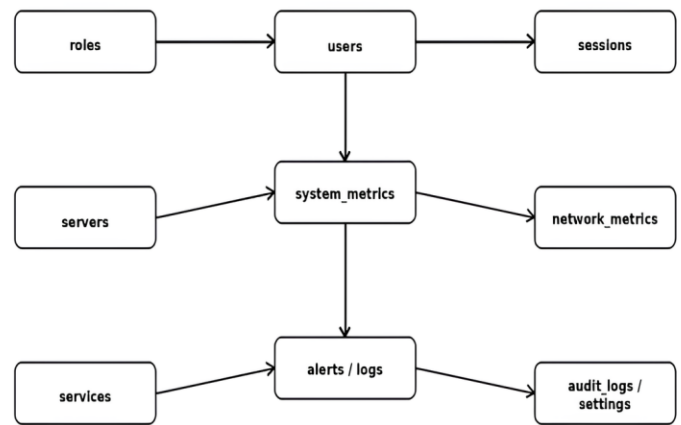


Fig. 3. Monitoring data flow from Linux host to dashboard.

IV. DATABASE DESIGN AND IMPLEMENTATION

PostgreSQL 16 is used as the relational persistence layer. The schema created during development contains alerts, audit_logs, logs, network_metrics, roles, servers, services, sessions, settings, system_metrics, and users. Seed data establishes Admin, Operator, and Viewer roles. The schema is intentionally broader than the currently displayed dashboard so that the same database can support monitoring history, security records, alerts, and configuration as the application grows.

ATLAS DATABASE - CONCEPTUAL RELATIONSHIP VIEW



Schema also includes persistent structures for monitoring, security events and application configuration.

Fig. 4. Conceptual relationship view of major ATLAS database entities.

A. User and Role Data

The users table stores ATLAS-specific user information while the roles table provides the basis for authorization. Firebase manages authentication identity, and verified users are synchronized into PostgreSQL. New synchronized users are associated with the Viewer role by default in the implemented repository design.

B. Monitoring and Operational Data

system_metrics and network_metrics provide persistence structures for collected telemetry. servers and services support infrastructure/service records. alerts, logs, and audit_logs provide a basis for operational and security event tracking, while settings stores configurable application values.

C. Database Integration Challenges

During implementation, PostgreSQL connectivity exposed practical configuration issues including a SCRAM password-type error and later sequence permission problems related to the users table. Correcting environment configuration and database privileges allowed the backend to connect successfully and supported the Firebase-to-PostgreSQL user synchronization flow. These debugging steps were an important part of validating the persistence layer rather than assuming that schema creation alone guaranteed correct application access.

V. AUTHENTICATION AND SECURITY

Because infrastructure dashboards can expose sensitive operational information, ATLAS includes authentication as a core boundary. The frontend authentication code supports email/password sign-in, registration, Google sign-in, password reset, persistence, and profile updates. Firebase Authentication performs the identity operation. The backend does not rely solely on browser state: Firebase Admin verifies the token supplied by the authenticated client.

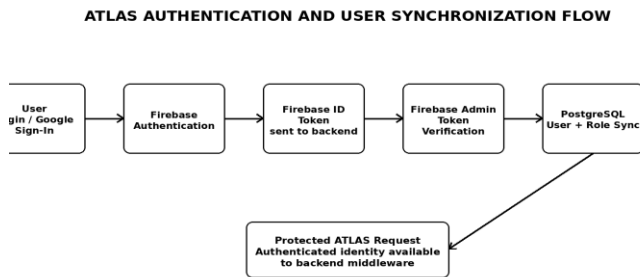


Fig. 5. Firebase authentication, backend verification, and PostgreSQL synchronization.

A. Backend Verification

Authentication middleware verifies the Firebase token and attaches decoded identity information to the request. The `/sync` route invokes the authentication controller and service, and the repository searches for a corresponding PostgreSQL user. Existing users can have login information updated, while new verified identities can be inserted into the ATLAS users table.

B. Role-Based Design

Admin, Operator, and Viewer roles are present in the database. This provides the foundation for role-based access control. Full enforcement of different permissions across every future administrative feature remains an extension of the current implementation and should not be confused with the completed identity-verification flow.

C. Security Practices

The backend uses environment-based configuration and includes Helmet, CORS, validation, and request logging dependencies. Firebase service-account credentials are stored separately from ordinary frontend code and should remain excluded from public source control. Authentication verification occurs on the server before protected operations depend on the supplied identity.

VI. MONITORING MODULE IMPLEMENTATION

The monitoring subsystem is the primary operational component of ATLAS. It is divided into `monitoringRoutes.js`, `monitoringController.js`, and `monitoringService.js`. This separation keeps HTTP routing, request coordination, and host data collection independent.

A. System Information

`GET /api/system` returns general host information including hostname, platform, architecture, uptime, Node.js version, CPU-core count, total memory, and free memory. During development the endpoint returned actual information from the WSL2 Ubuntu environment, confirming that ATLAS was reading the host rather than presenting static demonstration values.

B. CPU Monitoring

`GET /api/cpu` exposes processor-related information through the monitoring service. CPU data forms the basis for dashboard health indicators and can later be stored as historical telemetry or evaluated against alert thresholds.

C. Memory Monitoring

`GET /api/memory` provides memory-related information. The monitoring design supports total, used, and available memory representations depending on the service response and provides the foundation for high-memory alerts and historical analysis.

D. Disk Monitoring

`GET /api/disk` obtains storage information. Disk monitoring is important for identifying capacity exhaustion, which can affect databases, logs, applications, and the operating system itself.

E. Network Monitoring

`GET /api/network` exposes network-related data. Together with CPU, memory, disk, and general system information, it gives the dashboard a multi-resource view of host health.

F. Supporting Libraries

The implementation uses the Node.js `os` module, the `systeminformation` package, and Linux command execution where appropriate. Helper functions are used to format byte quantities and uptime into human-readable values.

VII. FRONTEND AND DASHBOARD

The frontend contains login.html, register.html, dashboard.html, CSS, JavaScript, configuration, assets, and supporting components/pages. Express serves the frontend as static content. The dashboard is designed to display CPU, memory, disk, network, system status, alerts, and other administrative information, with Chart.js available for visualization.

A. Login and Registration Interface

The authentication interface connects browser actions to Firebase Authentication. Email/password and Google authentication are supported, along with password reset and registration behavior. Successful authentication is followed by backend synchronization so that the identity can be represented in the ATLAS database.

B. Dashboard Integration

The monitoring APIs were individually verified to return values. Earlier development checkpoints showed the dashboard cards remaining in a loading state even while the API responses were valid, highlighting the distinction between backend monitoring correctness and frontend state/rendering integration. This report therefore treats verified API collection as implemented and avoids presenting unsupported frontend behavior as completed.

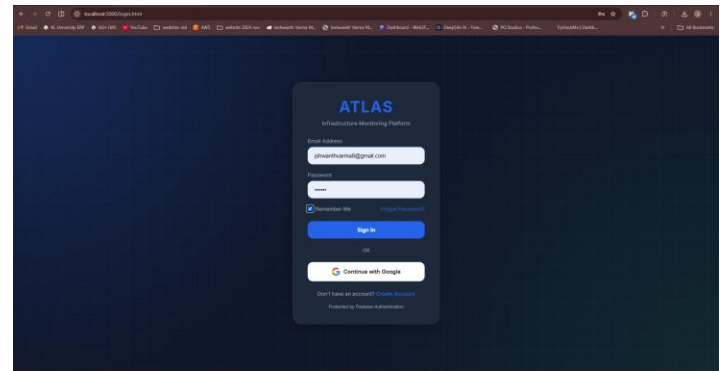


Fig. 6. ATLAS login interface with Firebase Authentication.

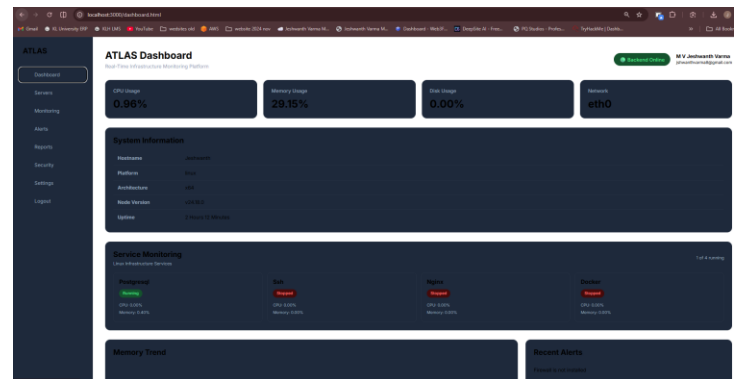


Fig. 7. ATLAS infrastructure monitoring dashboard displaying real-time system and service information.

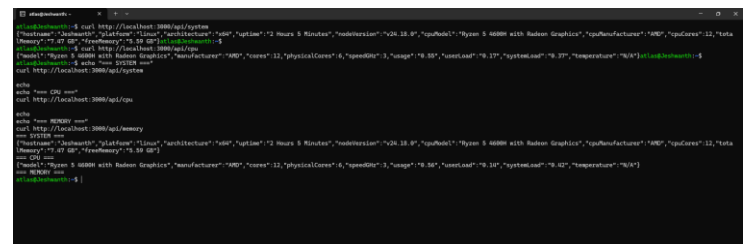


Fig. 8. Terminal output demonstrating REST API responses for real Linux system monitoring.

Test	Expected Result	Observed Result	Status
PostgreSQL connection	Backend connects to ATLAS database	Successful connection after configuration fix	Passed
Express startup	Server starts and serves frontend	Backend started and root/static CSS requests succeeded	Passed
GET /api/system	Return actual host information	Hostname/platform/architecture/uptime/memory returned	Passed
Monitoring APIs	CPU, memory, disk and network responses	Endpoints returned values during testing	Passed
Firebase login	Authenticate valid user	Successful login achieved	Passed
Backend token verification	Verify Firebase identity server-side	Middleware verification flow implemented and used	Passed
PostgreSQL user sync	Find/create authenticated ATLAS user	Implemented after privilege/sequence debugging	Passed
Dashboard metric rendering	Display API metrics in cards	Earlier checkpoint showed loading state	Needs final verification

VIII. TESTING AND RESULTS

Testing was performed incrementally throughout development. PostgreSQL service availability and database connectivity were checked before API development. The Express backend was started through the development workflow and confirmed to serve the application. Monitoring endpoints were tested directly and returned host information. Authentication testing covered Firebase login and backend verification, while database synchronization exposed and helped resolve PostgreSQL privilege issues.

A. Representative System Response

One recorded system-information response during development reported a Linux x64 host, Node.js v24.18.0, 12 CPU cores, approximately 7.47 GB total memory, and host uptime values generated at request time. Because these values are live measurements, exact free memory and uptime naturally vary between requests.

B. Result Discussion

The strongest result of the implementation is the successful connection of several normally separate concerns: Linux host data collection, an Express REST interface, Firebase identity, server-side token verification, PostgreSQL application records, and a browser frontend. The debugging process also demonstrated that end-to-end correctness depends on configuration and permissions across all layers, not only on individual source files.

IX. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

A. Advantages

ATLAS uses real host data instead of hard-coded monitoring values; separates major backend responsibilities into modular layers; combines authentication with application-level database records; uses a relational schema that can grow into historical monitoring and security analysis; and provides a single project through which Linux administration, web development, database design, authentication, and cybersecurity concepts can be demonstrated.

B. Current Limitations

The current project is primarily a single-host development implementation under WSL2. Complete remote-agent monitoring, production-grade deployment,

comprehensive role enforcement, long-term historical analytics, advanced alert automation, and full report generation are not claimed as finished. Dashboard rendering should be verified against the final build immediately before submission, especially because an earlier checkpoint showed cards remaining in a loading state while the APIs themselves were operational.

C. Future Scope

Future work can add Socket.IO-based push telemetry, persistent historical charts, configurable thresholds, email/browser alerts, richer audit analysis, service management, SSH and firewall monitoring, report export, remote Linux agents, Docker and Nginx deployment, automated backups, anomaly detection, and machine-learning-assisted identification of abnormal resource or security behavior. The modular architecture is intended to make these extensions possible without redesigning the entire application.

X. CONCLUSION

ATLAS demonstrates the design and implementation of a practical infrastructure monitoring and security platform using modern full-stack technologies. The project successfully integrates Ubuntu host monitoring, Node.js and Express REST APIs, PostgreSQL persistence, Firebase Authentication, backend token verification, and a browser-based frontend. Its monitoring endpoints provide actual system information for CPU, memory, disk, network, uptime, and host characteristics, while the authentication layer protects access and synchronizes verified identities with application records.

Beyond the visible dashboard, the project demonstrates important engineering concepts including separation of concerns, modular backend organization, environment configuration, relational schema design, API testing, privilege management, and debugging across multiple software layers. ATLAS therefore provides a strong foundation for future work in real-time monitoring, alerting, reporting, remote infrastructure management, and security analytics.

XI. ACKNOWLEDGMENT AND DECLARATION

The author expresses sincere gratitude to the faculty members of KL CDOE, VJA Campus, for the academic guidance and learning environment that supported the completion of this MCA project. The author also acknowledges the maintainers and documentation contributors of Ubuntu, Node.js, Express.js, PostgreSQL, Firebase, Chart.js, systeminformation, Git, and the other open-source technologies used during development.

Declaration

I, M. V. Jeshwanth Varma, Roll No. 24914120063, hereby declare that the project titled “ATLAS – Unified Infrastructure Monitoring and Security Platform” is my academic project work and that the implementation and report have been prepared for the requirements of the Master of Computer Applications program. External technologies, libraries, and documentation used in the project are acknowledged in the references. The report distinguishes implemented functionality from planned future extensions and does not intentionally present unimplemented features as completed work.

REFERENCES

- [1] Node.js Documentation, Node.js runtime and operating-system APIs.
- [2] Express.js Documentation, web application framework for Node.js.
- [3] PostgreSQL Documentation, PostgreSQL relational database system.
- [4] Firebase Documentation, Firebase Authentication.
- [5] Firebase Admin SDK Documentation, server-side authentication and token verification.
- [6] systeminformation Documentation, system and hardware information library for Node.js.
- [7] Chart.js Documentation, JavaScript charting library.
- [8] Ubuntu Documentation, Ubuntu Linux operating system.
- [9] Socket.IO Documentation, bidirectional event-based communication for web applications.
- [10] Git Documentation, distributed version control system.

APPENDIX A – IMPORTANT ATLAS API ENDPOINTS

/api/system – general host information

/api/cpu – CPU monitoring

/api/memory – memory monitoring

/api/disk – disk monitoring

/api/network – network monitoring

/api/auth/sync – authenticated user synchronization
(route naming based on auth route structure)

APPENDIX B – FINAL SUBMISSION SCREENSHOT CHECKLIST

Before submitting the college copy, replace the three marked screenshot placeholders with screenshots from the final running build: (1) login page, preferably showing the ATLAS branding; (2) dashboard with CPU, memory, disk, and network values visible; and (3) terminal or browser/API output proving the monitoring backend is returning real system information. Do not include Firebase service-account keys, database passwords, .env contents, tokens, or other credentials in screenshots.